

我觉得，这个消息其实比之前清晰太多了。

甚至可以说，你整个学习路线现在已经可以重新规划了。

因为你的方向已经不是泛 Android BSP，而是：

Android Display Stack（显示子系统）

这其实是 Android 最复杂的几个子系统之一（Camera、Display、Audio 基本属于第一梯队）。

第一性原理

我建议不要按照：

DRM → SurfaceFlinger → MIPI

这种模块学习。

而是按照：

一帧像素是如何最终显示到屏幕上的？

整个学习路线都围绕这个问题。

如果能回答：

APP draw 一个 Button，

为什么最后 OLED 会亮？

那么整个显示体系基本就通了。

我建议整个学习路线

可以分成八个阶段。

第一阶段（必须）

Display First Principle

问题只有一个：

一帧图像到底是什么？

学习：

Pixel

RGB

Color Space

Frame Buffer

Refresh Rate

Double Buffer

Triple Buffer

VSync

建议资料：

★★★★★

《Computer Graphics: Principles and Practice》

不用全看。

只看：

Rasterization

Framebuffer

Display

另外：

LearnOpenGL

里面关于 Framebuffer 那几章特别容易理解。

实践：

自己写一个 Linux Framebuffer Demo。

哪怕直接：

`/dev/fb0`

画一条红线。

你会第一次真正理解：

Frame Buffer。

第二阶段

Linux Display

开始进入 Kernel。

学习：

为什么 Linux 不直接操作 Framebuffer？

为什么后来变成：

DRM/KMS。

学习：

DRM

KMS

Plane

CRTC

Connector

Encoder

推荐：

★★★★★

Linux Kernel Documentation:

Documentation/gpu/drm/

这是官方文档。

然后:

LWN

有很多 DRM 系列文章。

实践:

把 DRM Pipeline 画出来。

例如:

Framebuffer

↓

Plane

↓

CRTC

↓

Encoder

↓

Connector

↓

Panel

以后天天都会看到。

第三阶段

Android Display

开始进入 Android。

学习：

Surface

SurfaceControl

BufferQueue

GraphicBuffer

Gralloc

SurfaceFlinger

HWComposer(HWC2)

Composer HAL

推荐：

★★★★★

Google 官方：

Graphics Architecture

BufferQueue

SurfaceFlinger

AOSP Graphics

这些官方文档非常好。

真正建议画：

APP

↓

Canvas/OpenGL/Vulkan

↓

BufferQueue

↓

SurfaceFlinger

↓

HWC

↓

DRM

↓

MIPI DSI

↓

Panel

以后所有问题都会在这里。

第四阶段

Buffer

这是 Android 最重要的。

重点：

为什么：

不能 memcpy?

学习：

DMA

DMA-BUF

Fence

Sync

ION

DMA Heap

Cache

IOMMU

推荐：

Linux dma-buf Documentation

Android Graphic Buffer

实践：

理解：

为什么 Camera 和 Display 可以共享 Buffer。

第五阶段

Display Driver

终于进入 Driver。

学习：

Panel Driver

Backlight

Bridge

DRM Driver

MIPI DSI Driver

Device Tree

Clock

Regulator

GPIO

Reset

Panel Timing

推荐：

Kernel：

drivers/gpu/drm/

里面：

panel/

bridge/

tiny/

都是优秀源码。

实践：

看一个 Panel Driver。

例如：

panel-boe-xxxx.c

看看：

prepare()

enable()

disable()

unprepare()

为什么存在。

第六阶段

MIPI DSI

这里开始真正接近 OEM。

学习：

DSI

DCS Command

Command Mode

Video Mode

Lane

Clock

Timing

LP

HS

推荐：

MIPI 官方公开资料。

另外：

NXP

TI

Rockchip

都有很多 Application Note。

重点：

理解：

为什么：

点屏失败？

为什么：

黑屏？

为什么：

花屏？

为什么：

闪屏？

为什么：

TE 不同步？

第七阶段

Boot Logo

你 Mentor 提到：

UEFI

Logo

其实很好。

启动流程：

BootROM

↓

UEFI

↓

Splash Logo

↓

Kernel

↓

DRM

↓

Android Boot Animation

学习：

UEFI GOP

Framebuffer

SimpleFB

SimpleDRM

Boot Logo

为什么：

Kernel 不闪？

为什么：

Logo 不丢？

第八阶段

Debug

以后天天干这个。

例如：

黑屏：

到底：

哪层坏了？

例如：

APP

↓

SurfaceFlinger

↓

HWC

↓

DRM

↓

DSI

↓

Panel

哪一步没数据？

或者：

花屏：

是不是：

Clock？

Lane？

Timing？

Memory？

Cache？

DMA？

Panel Init？

我觉得真正的学习路线应该这样

第一阶段

显示原理

★★★★★

↓

第二阶段

Linux DRM

★★★★★

↓

第三阶段

Android Graphics

★★★★★

↓

第四阶段

Buffer Management

★★★★★

↓

第五阶段

Display Driver

★★★★★

↓

第六阶段

MIPI DSI

★★★★★

↓

第七阶段

Boot Logo

★★★★☆

↓

第八阶段

Debug

★★★★★

以后工作中的业务

我觉得 Mentor 已经透露了很多。

以后你遇到的大概率就是：

例如：

黑屏

你需要判断：

是不是：

Panel 没初始化？

花屏

是不是：

MIPI Timing。

碎屏

是不是：

Memory Corruption。

闪屏

是不是：

VSync。

掉帧

是不是：

SurfaceFlinger。

Logo 卡住

是不是：

UEFI。

我建议你建立一张"显示知识地图"

以后所有知识都往这张图上挂，而不是零散学习：

应用层

| App (Canvas/OpenGL/Vulkan) |

Android Graphics

| Surface |
| BufferQueue |
| SurfaceFlinger |
| HWComposer(HWC2) |

HAL

| Composer HAL |
| Gralloc HAL |

Linux Kernel

| DRM/KMS |
| Plane / CRTC / Connector |
| Panel Driver |
| DMA-BUF / Fence |
| Device Tree / Clock / GPIO |

硬件

| MIPI DSI |
| Display Controller |
| Panel (LCD/OLED) |

最后，我有一个建议，我认为会让你未来一年受益最大。

结合你之前写 SoC 系列文章、DSP 系列文章的习惯，我建议你把学习目标定为：

不是学会 Android Display，而是写出一套《Android Display First Principle》系列。

原因很简单：当你能够把"一帧像素从 App 到 OLED"完整、准确地讲清楚时，说明你已经真正建立了系统级理解。按照你的岗位，我甚至建议把未来几个月的学习和输出都围绕这条主线展开，例如：

1. 为什么需要 Framebuffer?
2. 为什么 Linux 从 fbdev 演进到 DRM/KMS?
3. Surface、SurfaceFlinger、BufferQueue 到底解决了什么问题?
4. 为什么需要 Hardware Composer (HWC) ?
5. 一帧画面是如何零拷贝送到屏幕上的?
6. DRM/KMS 如何驱动一块 MIPI DSI OLED?
7. 从 UEFI Logo 到 Android Boot Animation 的完整启动显示链路。
8. 黑屏、花屏、闪屏、碎屏等典型问题如何分层定位。

这样学习会非常有方向感，也会与你未来的实际工作形成很强的正反馈。